Tuwaiq CLUB

أكاديمية طويق
Tuwaiq Academy

KFUPM
Student Affairs

DAY 1

Foundation

Setup, models, migrations, admin, ORM, and persistence

Sunday, October 25, 2026

Your five-day journey

One project grows all week: the Task Management API

01

Foundation

Project, Task model, migrations, admin

TODAY

02

Core API

CRUD with function views

03

Serialization & validation

Serializers, rules, clear field errors

04

Security & configuration

Tokens, ownership, environment

05

Publish

Deploy, verify, README, live demo

Today's session

About three hours: learn it, practice it, build it

Explain + live demo
~60 min

Exercise
~60 min

Break
short

Project
~60 min

Exit check
end of day

Priority rule: finish the exercise core, then move to the project. Stretch work can wait.

Learning outcomes

By the end of today you can...

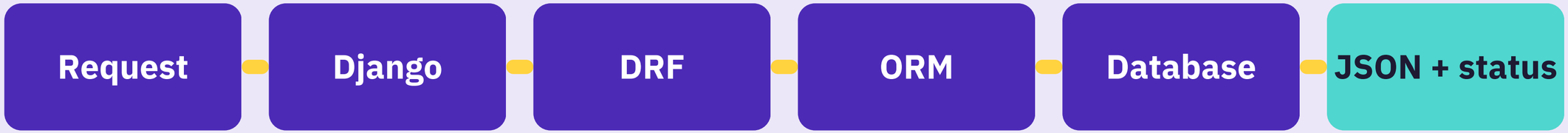
01 Explain the HTTP request–response flow

02 Run a reproducible Django REST Framework environment

03 Persist Task records through a correctly migrated data model

The finished project first

Start with the request, then trace what Django does with it




```
GET /api/tasks/7/   → 200 OK
{"id": 7, "title": "Write README", "status": "TODO"}
```

The URL names the resource. The verb names the action.

Client and server

The client asks. Your Django server decides what to do.

Client

Browser, Postman or mobile app
Sends an HTTP request

Server

Django reads the request,
uses the ORM and returns JSON

Request: method + URL + headers + body

Response: status code + headers + body

The client never touches the database. The server controls access.

Anatomy of HTTP

Read the method, URL, headers, body and status code

```
POST /api/tasks/create/ HTTP/1.1
Host: 127.0.0.1:8000
Content-Type: application/json
Authorization: Token <key>

{"title": "Prepare slides", "status": "TODO"}
```

```
HTTP/1.1 201 Created
Content-Type: application/json

{"id": 7, "title": "Prepare slides", "status": "TODO"}
```

Headers describe the message. A blank line separates them from the body. Tokens arrive on Day 4.

What is a web framework?

Django handles common web work; DRF adds tools for JSON APIs

Job	Django + DRF
URL to code	path() selects the view
Database access	Models + ORM produce SQL
Create tables	makemigrations + migrate
JSON in and out	request.data + Response; serializers on Day 3
Admin and identity	admin.site.register + users + token authentication

Write the behavior of your application; reuse the common web plumbing.

Databases 101

Tables contain rows; columns describe each row; keys connect records

id (PK)	Director
1	Christopher Nolan
2	Greta Gerwig

id (PK)	Movie	director_id (FK)
1	Inception	1
2	Oppenheimer	1
3	Lady Bird	2

Table = model class | Row = one object | Column = one field

Primary key: the unique identifier of a row.
Foreign key: a column holding another table's primary key.

Many movies belong to one director: each movie stores director_id.

SQL vs the ORM

The ORM writes SQL for you. These statements are simplified for reading.

ORM (Python)	SQL
`Movie.objects.all()`	`SELECT * FROM movies_movie;`
`Movie.objects.filter(genre="DRAMA")`	`SELECT * FROM movies_movie` `WHERE genre = 'DRAMA';`
`Movie.objects.create(title="Dune", ...)`	`INSERT INTO movies_movie (title, ...)` `VALUES ('Dune', ...);`
`movie.delete()`	`DELETE FROM movies_movie WHERE id = 3;`

```
print(Movie.objects.filter(genre="DRAMA").query)
```

HTTP verbs on the Task resource

One resource, separate URLs for each operation

Method	URL	Action	Success
GET	`/api/tasks/`	list	200
POST	`/api/tasks/create/`	create	201
GET	`/api/tasks/{id}/`	retrieve	200
PUT / PATCH	`/api/tasks/{id}/update/`	update	200
DELETE	`/api/tasks/{id}/delete/`	delete	204

400 invalid input | 401 no token (Day 4) | 404 not found

Project vs app

How a Django codebase is organized

```
task-management-api/
├── config/   # the project
│   ├── settings.py
│   └── urls.py
├── tasks/    # the app
│   ├── models.py
│   └── admin.py
└── manage.py
```

config is the project

Settings, root URLs, and the WSGI entry point

tasks is the app

Models and admin today; views and URLs from Day 2

Django provides models, the ORM, admin, and users. DRF sits on top and turns them into an API.

Model → migration → database

Django turns Python model changes into real tables

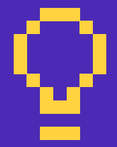


- Change a model, then run both commands again
- makemigrations writes a new numbered file and never edits old ones
- Migrations are committed to Git
- db.sqlite3 is rebuilt locally and is not committed

Field options and relationships

These rules appear in both the example and the exercise

Option	Meaning
`max_length`	required for CharField
`blank=True`	may be left empty in forms and validation
`null=True`	database may store NULL
`default=...`	value used when none is given
`unique=True`	no two rows share the value
`ForeignKey`	one parent can have many related rows



LIVE DEMO

Movies worked example

15 min: build it and show the table in DB Browser

models.py	**migrations**	**admin.py**	**ORM**
fields, choices, __str__	makemigrations + migrate	register and customize	create, read, update, delete

Afterwards, the Movies files are your read-only syntax reference

Reproducible setup

The environment should work the same way for every participant

```
# Windows
.venv\Scripts\Activate.ps1
# macOS
source .venv/bin/activate

python manage.py check
python manage.py runserver
```

- requirements.txt freezes the shared stack
- .gitignore keeps .venv, db.sqlite3 and .env out of Git
- check confirms Django can load the project
- A fresh terminal should start the server

Today is local. Production comes on Day 5.

HOUR 2 · EXERCISE

Library Catalog

Models, migrations, admin, and ORM practice

Core	Stretch	Time
Tasks 1, 2, 3A	Task 3B	~45 minutes

Result: 3 authors, 6 books, and a working admin

`Start in days/day-01/exercise/`

Exercise Task 1: complete Book

Use the existing Author model as your pattern

Field	Type	Rule
`title`	CharField	max_length=200
`isbn`	CharField	max_length=13, unique=True
`published_year`	PositiveIntegerField	required
`available_copies`	PositiveIntegerField	default=1
`author`	ForeignKey → Author	CASCADE, related_name=books

Also complete __str__ so a Book displays as its title.

Exercise Task 2: create the database

No code to write. Run the commands and read their output.

```
python manage.py makemigrations catalog
python manage.py migrate
python manage.py loaddata sample_data
```

- 0001_initial.py creates Author and Book
- migrate creates db.sqlite3 and tables
- loaddata installs 9 objects
- Optional: inspect catalog_book in DB Browser

Close DB Browser before running Django again, otherwise the database may be locked.

Exercise Task 3: admin and ORM

Finish the core admin task before the stretch queries

Core 3A · Admin
5 columns · author filter · title / ISBN search

Stretch 3B · ORM
count · filter · lookups · create · update · delete
Goal: ORM practice 5/5 passed

- Core acceptance: both models appear in admin
- Searching “clean” shows 3 books
- Filtering Martin Fowler shows 2 books

Short on time? Do the stretch after the project milestone.

BREAK

Short break

After the break: start your own Task Management API

HOUR 3 · PROJECT

Start the Task API

Apply the same ideas to your own repository · about 60 minutes

Create from the starter
new folder + new GitHub repo

Complete P1–P5
settings, model, __str__, admin

Build and verify
users, tasks, restart, push

Setup: create the project .venv and install requirements.txt.
Push: git init → check git status → add and commit → set origin → push main. Keep .venv/, db.sqlite3, and secrets out of Git.

Day 1 project increment

Five TODOs in three files

TODO	File	Purpose
P1	`config/settings.py`	register rest_framework and tasks
P2–P4	`tasks/models.py`	status choices, fields, __str__
P5	`tasks/admin.py`	columns, filters, search

Then run makemigrations and migrate, add two users and three tasks in admin, restart, and push.

The Task table

Build the data foundation for the API

Field	Type	Rule
`title`	CharField(200)	required
`description`	TextField	blank=True
`status`	CharField(20)	TODO / IN_PROGRESS / DONE
`due_date`	DateField	null=True, blank=True
`owner`	ForeignKey → User	CASCADE, related_name=tasks
`created_at / updated_at`	DateTimeField	auto_now_add / auto_now

In admin: create two users and at least three tasks with different owners and statuses.

Day 1 exit check

You are ready for Day 2 when all five pass

01	A fresh terminal activates .venv and starts the server
02	migrate reports “No migrations to apply”
03	Task fields and status choices match the table
04	Admin keeps records after a restart
05	GitHub has no .venv, db.sqlite3, or secrets

First push ran over? Finish it in the first 10 minutes of Day 2.

Tuwaiq CLUB

أكاديمية طويق
Tuwaiq Academy

KFUPM
Student Affairs



Day 1 complete

You built the data foundation: app, model, migrations, admin, and saved records.

Next: function views, path() URLs, and the CRUD endpoints